

Lab 1: Simulating Simple LLM Inferences

In this lab, you send requests to a simulated LLM server and measure how long they take. You will be using the Vidur-Agent simulator [1], which was developed from our lab, by extending the Vidur [2] simulator from Microsoft.

You generate inputs as a simple CSV file: which requests arrive, when, with how long a prompt and how many output tokens. The simulator tells you when every output token came out. Your job is to design experiments, run them, and explain what you see, with respect to what we learned in class.

[1] Donghwan Kim, Prakhar Singh, Younghoon Min, Jongryool Kim, Jongse Park, and Kiwan Maeng. "Characterizing How Complex Agentic AI Systems Handle General Tasks: A Trace-Based Simulation Study." IISWC 2026.

[2] Amey Agrawal, Nitin Kedia, Jayashree Mohan, Ashish Panwar, Nipun Kwatra, Bhargav S. Gulavani, Ramachandran Ramjee, and Alexey Tumanov. "Vidur: A Large-scale Simulation Framework for LLM Inference." MLSys 2024.

Policy on AI usage

You can get help from AI to write code, interpret results, generate plots, etc., but **you must be responsible for all the code and results**. You are subject to being interviewed, and you may not get full credit if you cannot explain your code and/or your results properly. **All the writings in the report must be of your own and must not be AI-generated.**

The system you are simulating

- **Model:** Qwen2.5-32B.
- **Hardware:** 4 NVIDIA A100 GPUs working together on every request (tensor-parallelism degree 4).
- **Steps:** the server runs in **steps**. Each step processes a whole or partial prompt (*prefill*), and/or generate one new output token (*decode*). Which tokens go into which step is the scheduler's decision.
- **Memory:** We assume at most 256 requests can run at once, and no prefix caching is turned on.

Part 1 — Setup

You need Python 3.10+ (Westgate W135 has 3.11) and at least about 1.5 GB of disk.

0. Log into Westgate W135 machine, either physically or remotely.

Follow <https://www.eecs.psu.edu/cse-student-lab-access/index.aspx>. If you are not using Penn State Wifi, you need a VPN for remote access.

Alternatively, it is okay to work with any other machine you'd like. The lab does not require any GPU access. However, if you choose to do so, you are responsible for setting up your own environment. In case you have any technical issues, I will not be able to help. All the below setup instructions were tested on a W135 machine.

1. Set up a virtual environment in scratch space

Make a folder in scratch space (Replace [xyz1234] with your Penn State access ID.) We are using the scratch folder because the home directory has very limited space. Please note that the scratch folder is not synced across different machines, so it is recommended to use a single machine for this project.

```
mkdir -p /scratch/[xyz1234]/Lab1
cd /scratch/[xyz1234]/Lab1
python -m venv .venv
source .venv/bin/activate
```

2. Download and install Vidur-Agent

Clone Vidur-Agent from its GitHub.

```
git clone git@github.com:psu-paws/Agent-AE.git
cd Agent-AE/Vidur-Agent
python -m pip install -e .
```

Everything below is run from this `Vidur-Agent/` directory.

Then, download `lab1.zip` from Canvas and unzip it under `lab1/`. Under `Vidur-Agent/`,

```
unzip lab1.zip
```

3. Install the performance predictor

The simulator uses a trained predictor to predict the performance of the GPU for running each prefill and decode step. You must install a pre-trained predictor. Download the `cache_lite_qwen32b_a100_tp2_tp4.tar.gz` file from Canvas, and place it under `Vidur-Agent/cache/`:

```
mkdir -p cache
cp /PATH-OF-THE-DOWNLOADED-FILE/cache_lite_qwen32b_a100_tp2_tp4.tar.gz cache/
python lab1/link_cache.py
```

The compressed file is around 190 MB, and around 1.5 GB when uncompressed (make sure you have enough space!).

4. Check it works

```
python lab/run_lab.py lab/inputs/example.csv
```

You should see a table of the simulated execution results of three requests. If instead you get "Prediction tables missing", step 3 did not finish; re-run it.

Part 2 — How the scheduler works

The simulated scheduler is from [vLLM](#) v1. It is not exactly the same with any of the scheduler we learned (Orca, vLLM, or Sarathi-serve) but it is similar enough. In each step it:

- Generates **one** new token for every request that is in its decode stage.
- Run (chunked) **prefill** at the same time, when there is a new request that needs its prefill stage to run.
- The chunk size parameter (`--chunk-size` , 4096 by default) controls how many prefill tokens you can run on each step. If you have `d` decode requests, you can run at most `chunk_size - d` number of tokens of prefill. If the prefill and decode length are both much smaller than the chunk size, this works similarly to Orca. If prefill is longer than chunk size, this works similarly to

Sarathi-serve.

Limits: Only prompts (prefill length) up to 4096 tokens are supported. This is not a limitation of the simulator itself; this is just because the performance predictor I gave you (which is 1.5GB) is not complete (a complete predictor is much larger).

Terms

For one request:

Term	Meaning	How the tool computes it
Prefill	Processing the whole prompt. It ends with the first output token	
Decode	Generating the remaining output tokens, one per step	
Queue time	Waiting before the request's first step	first step start – arrival
TTFT	Time to first token	first token time – arrival
TPOT	Time per output token, after the first	(last token time – first token time) / (output_len – 1)
E2E	End-to-end latency	last token time – arrival
Token gap	Time between two consecutive output tokens of one request. TPOT is their <i>average</i>	per token, in <code>tokens.csv</code>

Part 3 — Writing inputs and running

Input format

One CSV file per run, one row per request. Lines starting with `#` are comments.

```
# lab/inputs/my_first.csv
arrival_time,prompt_len,output_len,count
0.0,1024,128,1
0.5,512,256,2
```

Column	Meaning
<code>arrival_time</code>	when the request arrives, in seconds from the start
<code>prompt_len</code>	prompt length in tokens (1–4096)
<code>output_len</code>	how many tokens to generate
<code>count</code>	<i>optional</i> : repeat this row that many times (default 1)

That file means: one request with a 1024-token prompt at time 0, and two identical requests with 512-token prompts half a second later. Requests are numbered 0, 1, 2, ... in file order (after expanding `count`). Put your files in `lab/inputs/`.

The tool checks your file before running and tells you what is wrong (missing column, a prompt over 4096, and so on).

Running

```
python lab/run_lab.py lab/inputs/YOUR_INPUT.csv
```

Options:

- `--chunk-size N` sets the per-step token budget, 1 to 4096 (default 4096).
- `--out DIR` writes results somewhere other than `lab/results/`.
- Several files at once: `python lab/run_lab.py lab/inputs/exp1_*.csv`. All the files in the path starting with `exp_1` and ending with `.csv` (`*` is a wildcard; matches any string) run one after another.

Part 4 — Reading the results

The run prints one line per request. For more detail, each input file gets a folder `lab/results/<file name>/` with three CSVs:

requests.csv — one row per request

Column	Meaning
arrival_s , prompt_len , output_len	describes your input request arrival time, prompt (prefill) length, and output (decode) length
queue_ms	how long it waited before its first step
ttft_ms	time to first token
tpot_ms	average time per output token after the first
e2e_s	time between request arrival until the last token generation finishes
worst_gap_ms , worst_gap_token	the longest wait between two of its tokens, and which token
restarts	times it was evicted and recomputed (normally 0)

tokens.csv — one row per generated token

Column	Meaning
request , token	which request, and which token (1 = first)
time_s	when that token came out
gap_ms	time since that request's previous token (for token 1, since it arrived — so that is the TTFT)
step	which server step produced it

steps.csv — one row per server step

Column	Meaning
<code>step</code> , <code>start_s</code> , <code>end_s</code> , <code>duration_ms</code>	when the step ran and how long it took
<code>batch_size</code>	how many requests were in it
<code>prefill_tokens</code> , <code>decode_tokens</code>	prompt tokens and output tokens processed
<code>requests</code>	which requests were in it

`tokens.csv` and `steps.csv` share the `step` column, so for any token you can look up the step that produced it and see what else that step was doing. That is the main way to explain a slow token.

The `vidur/` subfolder holds the simulator's raw output; you do not need it.

A worked example

From `lab/inputs/example.csv` (the file shown above), the printed summary would look something like:

file	request	arrival_s	prompt_len	output_len	queue_ms	ttft_ms	tpot_ms	e2e_s	worst_gap_ms
example.csv	0	0.0	1024	128	0.00	113.73	32.77	4.28	116.39
example.csv	1	0.5	512	256	0.21	116.60	32.13	8.31	32.18
example.csv	2	0.5	512	256	0.21	116.60	32.13	8.31	32.18

Read it like this:

- Request 0's 1024-token prompt took **113.7 ms** to process (TTFT), then it produced 128 tokens at **32.8 ms** each, finishing 4.28 s after it arrived.
- Requests 1 and 2 arrived at 0.5 s, waited **0.21 ms** for the step already running, then were prefilled together in one step.
- Request 0 has one token that took **116.4 ms** instead of ~32 ms (Why do you think this was the case? Look it up in `tokens.csv` (the row with the large `gap_ms`), take its `step`, and find that step in `steps.csv` to see what happened!).
- Note that request 0's **TPOT barely moved** (32.8 vs 32.1 ms) even though one token took 3.6× as long, because TPOT shows an average across all tokens.

Part 5 — Experiments

Write your own input files. **Don't manually create input files. Use a script (you can use your favorite language!).** Write a single report that contains all the answers to each question (label them nicely so that I can see which answers are for which questions). There's a lot of questions. Your report must contain answers to all the questions, and plots when needed.

Warm-up 1: Prompt length vs. latency (one request)

Send a **single** request. Keep `output_len` fixed to 128; increase `prompt_len` from small up to 4096. Use the default chunk size (4096).

- How does TTFT grow with prompt length? Why? Draw a plot (`prompt_len` vs TTFT).
- Does TPOT depend on prompt length? Why or why not? Draw a plot (`prompt_len` vs TPOT).
- From TTFT and TPOT, how can you calculate the end-to-end execution time? Does it match the reported `e2e_s`?
- Draw a plot showing the percentage breakdown between prefill and decode latency for the different prompt length. Between prefill and decode, which one is the bottleneck?

Warm-up 2: Output length vs. latency (one request)

Send a **single** request. Keep `prompt_len` fixed to 2048; increase `output_len`. Use the default chunk size (4096).

1. Does TTFT grow with output length? Why or why not? Draw a plot (`output_len` vs TTFT).
2. How does TPOT depend on output length? Why? Draw a plot (`output_len` vs TPOT).
3. From TTFT and TPOT, how can you calculate the end-to-end execution time? Does it match the reported `e2e_s`?
4. Draw a plot showing the percentage breakdown between prefill and decode latency for the different output length. Between prefill and decode, which one is the bottleneck?

Experiment 1: Batch size vs. latency

Send **N identical requests that all arrive at time 0** (use `count`). Keep `prompt_len=16` and `output_len` fixed to some value; increase N from 1 to 256.

1. How does TTFT change with N? Why? Draw a plot (N vs. TTFT)
2. How does TPOT change with N? Why? Draw a plot (N vs. TPOT)

Experiment 2: New requests arrive!

Build this up in stages. Keep the first request the same throughout: a moderate prompt and a **long** output, arriving at time 0. This part is more open-ended. I will not tell you exactly what to discuss, and what plots to draw. At your liberty, write the report with your best effort.

4a. One request alone. Run just the first request. This is your baseline.

4b. Add a second request. Add a second request that arrives **while the first is still generating output**. Compare the first request's TPOT and token gaps with 4a. Why did they change? Use `tokens.csv` and `steps.csv` to exactly pinpoint what happened.

4c. Vary the second request's prompt length. Keep everything else fixed and sweep the second prompt from a few hundred tokens up to 4000. For each, discuss how the first request's worst token gap, and its TPOT changes.

4d. Vary the prefill chunk size. Repeat 4c at several chunk sizes, e.g. 4096, 2048, 1024, 512 and 256. Discuss what happens.

What to hand in

A single report in `.pdf` format, all your input files (you must include the **exact input files** used for your experiments), and your script to generate the input files. Also, describe briefly in your report how to run your input generation script. If you wrote any additional scripts/code, also attach them. **Zip everything into a single file** and upload to Canvas.

If something goes wrong

Message	Fix
Prediction tables missing from cache/	Re-run Part 1 step 3: the tarball must be in <code>cache/</code> , then <code>python lab/link_cache.py</code>
<code>cache/cache_lite_...tar.gz</code> not found	Put the file your instructor gave you in <code>cache/</code>
<code>prompt_len</code> must be ≤ 4096	Shorten the prompt; the measurements only cover prefills up to 4096 tokens
<code>prompt_len + output_len</code> must be ≤ 32768	The model's context window
missing column(s)	Your CSV needs <code>arrival_time,prompt_len,output_len</code> (and optionally <code>count</code>)
<code>ModuleNotFoundError: vidur</code>	Re-run Part 1 step 4